LAPORAN PRAKTIKUM MIKROKONTROLER

KONVERSI DATA DESIMAL KE OKTAL

BAHASA ASSEMBLY



Elita Lestari Setianto NIM 201344008

Kelas 3 NK

Tanggal Percobaan : Rabu, 07 Agustus 2022

Tanggal Penyerahan Laporan : Rabu, 14 September 2022

Instruktur : 1. Ferry Satria, BSEE, M.T.

2. Rahmawati Hasanah, S.ST., M.T.

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2022

A. TUJUAN

1. Untuk memenuhi tugas mata kuliah Mikrokontroler.
2. Mahasiswa dapat mengetahui bahasa pemograman Assembly.
3. Mahasiswa mampu melakukan instalasi dan menggunakan *software* AVR Studio 4 untuk simulasi pemrogaman mikrokontroler.
4. Mahasiswa mampu memahami konsep Bahasa Pemrograman Assembly dan menjalankannya di *software* AVR Studio 4.
5. Mahasiswa mampu memahami konsep dan membuat program input data serta konversi data dari decimal ke oktal dan sebaliknya pada *software* AVR Studio 4.

B. PENDAHULUAN

1. Mikrokontroler

Mikrokontroler merupakan suatu terobosan teknologi mikroprosesor dan mikrokomputer yang merupakan teknologi semikonduktor dengan kandungan transistor yang lebih banyak namun hanya membutuhkan ruang yang sangast kecil, Lebih lanjut, mikrokontroler merupakan system computer yang mempunyai satu atau beberapa tugas yang sangat spesifik, berbeda dengan PC (*Personal Computer*) yang memiliki beragam fungsi.

Tidak seperti sistem komputer yang mampu menangani berbagai macam program aplikasi, mikrokontrler hanya bisa digunakan untuk suatu aplikasi tertentu saja, perbedaan lainnya terletak pada perbandingan RAM dan ROM. Pada sistem komputer perbandingan RAM dan ROM nya besar, artinya program-program penggunba disimpan dalam ruang RAM yang relative besar, sedangkan rutin-rutin antar muka perangkat keras disimpan dalam ruang ROM yang kecil, Sedangkan pada mikrokontroler, perbandingan ROM dan RAM â€“nya yang besar, artinya program kontrol disimpan dalm ROM (bias *Masked* ROM atau *Flash* PEROM) yang ukurannya relatif lebih besar, sedangkan RAM digunakan sebagai tempat penyimpanan sementara , termasuk register-register yang digunakn pada mikrokontroler yang bersangkutan.

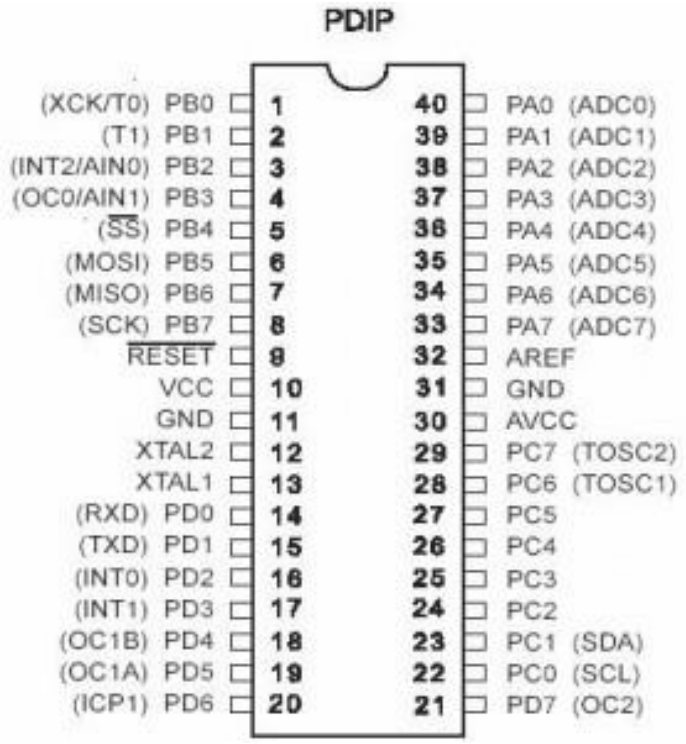
2. **Mikrokontroler ATMega8535**

Mikrokontroler merupakan keseluruhan sistem komputer yang dikemas menjadi sebuah *chip* di mana di dalamnya sudah terdapat Mikroprosesor, I/O, Memori bahkan ADC, berbeda dengan Mikroprosesor yang berfungsi sebagai pemroses data (Heryanto, dkk, 2008:1).

Mikrokontroller AVR (*Alf and Vegardâ€™s Risc processor*) memiliki arsitektur 8 bit, dimana semua instruksi dikemas dalam kode 16-*bit* dan sebagian besar instruksi dieksekusi dalam 1 siklus *clock* atau dikenal dengan teknologi RISC (*Reduced Instruction Set Computing*). Secara umum, AVR dapat dikelompokan ke dalam 4 kelas, yaitu keluarga AT90Sxx, keluarga ATMega dan AT86RFxx. Pada dasarnya yang membedakan masing-masing adalah kapasitas memori, *peripheral* dan fungsinya (Heryanto, dkk, 2008:1). Dari segi arsitektur dan instruksi yang digunakan, mereka bisa dikatakan hampir sama. Berikut ini gambar Mikrokontroler Atmega8535.



Gambar 2.1 *Mikrokontroler ATMega8535*



Gambar 2.2 Konfigurasi Pin ATMega8535

Â **Konfigurasi Pin ATMega8535**

Secara umum konfigurasi dan fungsi pin ATMega8535 dapat dijelaskan sebagai berikut

1 **VCC** Input sumber tegangan (+)

2 **GND** Ground (-)

3 **Port A (PA7 â€¦ PA0)** Berfungsi sebagai input analog dari ADC (Analog to Digital Converter). Port ini juga berfungsi sebagai port I/O dua arah, jika ADC tidak digunakan.

4 **Port B (PB7 â€¦ PB0)** Berfungsi sebagai port I/O dua arah. Port PB5, PB6 dan PB7 juga berfungsi sebagai MOSI, MISO dan SCK yang dipergunakan pada proses downloading. Fungsi lain port ini selengkapnya bisa dibaca pada buku petunjuk â€• AVR ATMega8535â€• .

5 **Port C (PC7 â€¦ PC0)** Berfungsi sebagai port I/O dua arah. Fungsi lain port ini selengk apnya bisa dibaca pada buku petunjuk â€• AVR ATMega8535â€• .

6 **Port D (PD7 â€¦ PD0)** Berfungsi sebagai port I/O dua arah. Port PD0 dan PD1 juga berfungsi sebagai RXD dan TXD, yang dipergunakan untuk komunikasi serial. Fungsi lain port ini selengkapnya bisa dibaca pad a buku petunjuk â€• AVR ATMega8535â€• .

7 **RESET** Input reset.

8 **XTAL1** Input ke amplifier inverting osilator dan input ke sirkuit clock internal.

9 Â Â **XTAL2** Output dari amplifier inverting osilator.

10 **AVCC** Input tegangan untuk Port A dan ADC.

11 **AREF** Tegangan referensi untuk ADC.

Fitur Mikrokontroler ATMega8535

Adapun kapabilitas detail dari ATmega8535 adalah sebagai berikut,

1 Sistem mikroprosesor 8 bit berbasis *RISC* dengan kecepatan maksimal 16 MHz.

2 Kapabilitas memori *flash* 8 KB, *SRAM* sebesar 512 byte, dan *EEPROM* (*Electrically Erasable Programmable Read Only Memori*) sebesar 512 byte.

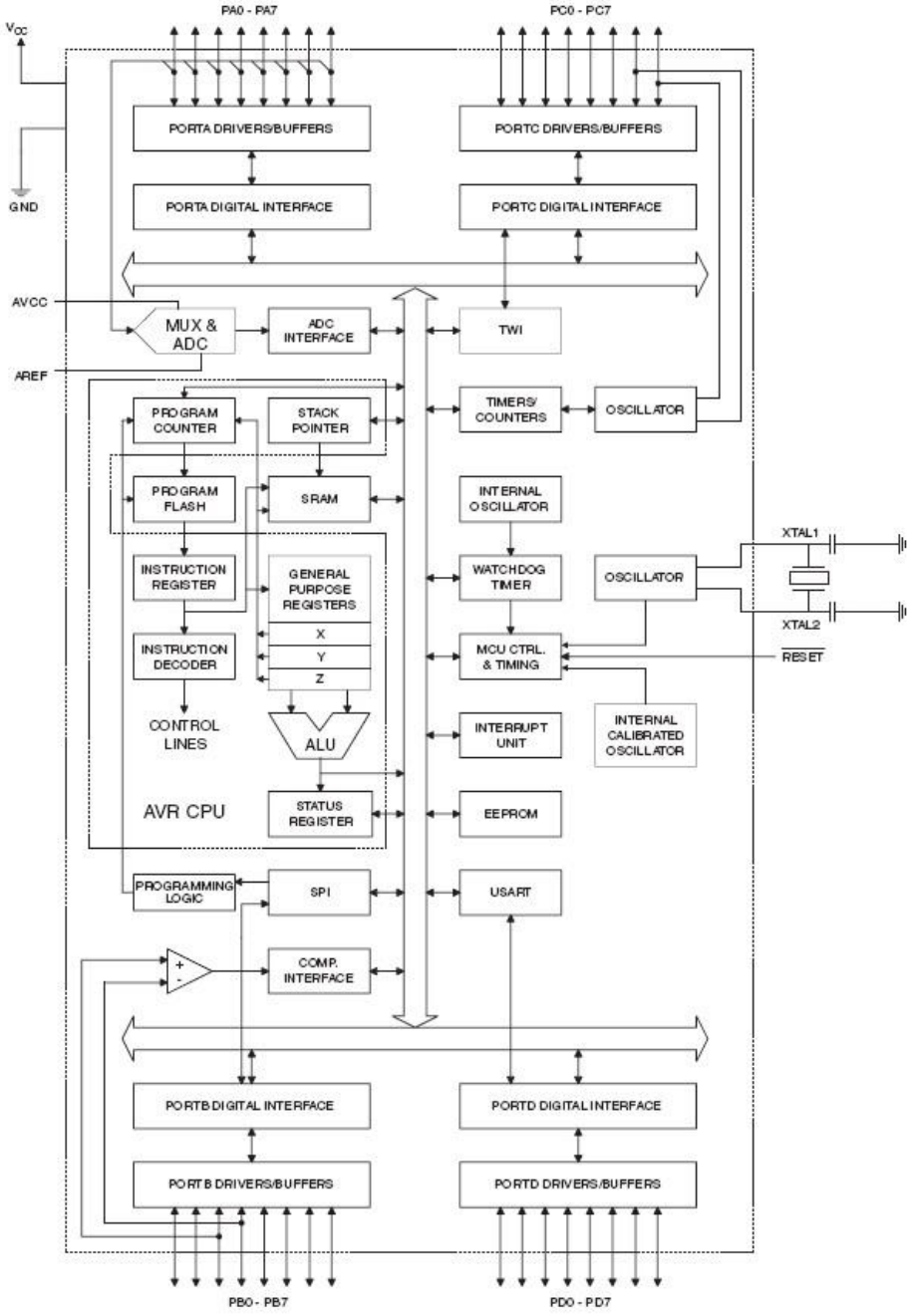
3 *ADC* internal dengan fidelitas 10 bit sebanyak 8 *channel*.

4 Portal komunikasi serial (*USART*) dengan kecepatan maksimal 2,5 Mbps.

5 Enam pilihan mode *sleep* untuk menghemat penggunaan daya listrik.

Arsitektur ATMega8535

Blok-Diagram



Gambar 2.3 *Blok diagram fungsional ATmega8535*

Dari gambar blok diagram tersebut dapat dilihat bahwa ATMega8535 memiliki bagian-bagian sebagai berikut :

1. Saluran I/O sebanyak 32 buah, yaitu *Port* A,*Port* B,*Port* C dan *Port* D.

2. ADC 8 *channel* 10 bit.

3. Tiga buah *Timer*/*Counter* dengan kemampuan pembanding.

4. CPU yang terdiri atas 32 buah *register*.

5. *Watchdog timer* dengan osilator *internal*.

6. SRAM sebesar 512 byte.

7. Memori *Flash* sebesar 8 KB dengan kemampuan *Read While Write*.

8. *Interrupt internal* dan *eksternal*

9. *Port* antarmuka SPI (*Serial Peripheral Interface*).

10. EEPROM sebesar 512 byte yang dapat diprogram saat operasi.

11. Antarmuka komparator analog.

12. *Port* USART untuk komunikasi serial

3. Software AVR Studio

Software CodeVisionAVR merupakan perangkat lunak kompailer C dan merupakan pembangkit program otomatis. Software CodeVisionAVR didesain untuk mikrokontroller keluarga atmel AVR. Software CodeVisionAVR dapat berjalan pada sistem operasi windows 95, 98, Me, NT 4, 2000 dan windows XP. Kompailer C melakukan pendekatan elemen dari bahasa ANSI C sesuai dengan arsitektur dari AVR, dengan beberapa fitur yang ditambahkan. Integrated Development Environment (IDE)

mendukung software AVR Chip In-System Programmer sehingga memungkinkan untuk melakukan transfer secara otomatis dari program ke chip mikrokontroller setelah berhasil dikompilasi atau diassembly. Software InSystem Programmer didesain untuk dapat bekerja dan mendukung tipe chip AVR STK500/AVRISP/AVRProg, Kanda Systems STK200+/300, Dontronics DT006, Vogel Elektronik VTEC-ISP, Futurlec JRAVR dan MicroTronics ATCPU/Mega2000 programmers/development boards. Software CodeVisionAVR juga mempunyai fasilitas CodeWizardAVR, yang digunakan untuk membantu menuliskan dengan cepat secara otomatis kedalam program. Dengan CodeWizardAVR membantu kita untuk membuat kerangka program dan menuliskan fungsi-fungsi:

A. Mengatur External memory
B. Mengidentifikasi sumber reset Chip
C. Inisialisai Port Input/Output8]
D. Inisialisasi External Interrupts
E. Inisialisasi Timers/Counters
F. Inisialisasi Watchdog Timer
G. Inisialisasi UART (USART) and interrupt driven buffered serial communication
H. Inisialisasi Analog Comparator
I. Inisialisasi ADC
J. Inisialisasi antarmuka SPI
K. Inisialisasi antarmuka Two Wire
L. Inisialisasi antarmuka CAN
M. Inisialisasi I2C Bus, LM75 Temperature Sensor, DS1621 Thermometer/Thermostat dan PCF8563, PCF8583, DS1302, DS1307 Real Time Clocks
N. Inisialisasi 1 Wire Bus and DS1820, DS18S20 Temperature Sensors
O. Inisialisasi LCD module

4. Konversi Data

Konversi data merupakan pengubah format data komputer dari satu format ke format lainnya yang akan di data lalu dikodekan atau dioperasikan dengan berbagai cara serta dibuat berdasarkan standar tertentu.[1] Ada berbagai cara untuk mengonversi suatu dokumen

1. Desimal ke Biner

Cara mengkonversi bilangan desimal ke biner adalah dengan cara membagi bilangan desimal dengan 2 (basis bilangan biner) dan menyimpan sisa hasil bagi dari setiap pembagian sebagai bit-bit bilangan biner. Nilai konversinya adalah urutan sisa hasil bagi dari yang paling akhir.

Contoh :

$88_{(10)} = \ldots_{(2)}$

- 88 : 2 = 44 sisa 0
- 44 : 2 = 22 sisa 0
- 22 : 2 = 11 sisa 0
- 11 : 2 = 5 sisa 1
- 5 : 2 = 2 sisa 1
- 2 : 2 = 1 sisa 0
- 1 : 2 = 0 sisa 1

Maka hasil Konversinya = 1011000 (ditulis dengan urutan dari bawah ke atas)

2. Desimal ke Octal

Cara mengkonversi bilangan desimal ke Oktal adalah dengan cara membagi bilangan desimal dengan 8 (basis bilangan oktal) dan menyimpan sisa hasil bagi dari setiap pembagian sebagai bit-bit bilangan oktal. Nilai konversinya adalah urutan sisa hasil bagi dari yang paling akhir.

Contoh :

1402(10) = … (8)

- 1402/8 = 175 sisa 2
- 175/8 = 21 sisa 7
- 21/8 = 2 sisa 5

· 2/8 = 0 sisa 2

Maka hasil konversinya = 2572 (ditulis dari bawah)

3. Desimal ke Hexadecimal

Cara mengkonversi bilangan desimal ke hexadesimal adalah dengan cara membagi bilangan desimal dengan 16 (basis bilangan hexadesimal) dan menyimpan sisa hasil bagi dari setiap pembagian sebagai bit-bit bilangan heksadesimal. Apabila sisa bagi > 9 maka angkanya dirubah menjadi huruf. Untuk sisa bagi berjumlah 10 = A, 11 = B, 12 = C, 13 = D, 14 = E, 15 = F.

Contoh :

19889(10) = … (16)

· 19889/16 = 1243 sisa 1

· 1243/16 = 77 sisa 11 (B)

· 77/16 = 4 sisa 13 (D)

· 4/16 = 0 sisa 4

Maka hasil konversinya = 4DB1 (ditulis dari bawah)

TABEL KONVERSI BILANGAN

Decimal (x_{10})	Biner (x_2)	Octal (x_8)	Hexa (x_{16})
0	0000	0	0
1	0001	1	1
2	0010	2	2
3	0011	3	3
4	0100	4	4
5	0101	5	5
6	0110	6	6
7	0111	7	7
8	1000	10	8
9	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

4. Biner ke Octal

Cara mengkonversi bilangan biner ke oktal yakni dengan mengelompokan bilangan biner menjadi 3 kelompok **dimulai dari bilangan biner yang paling kanan**. Setelah dikelompokan barulah kita dapat mengkonversi menjadi bilangan Oktal.
Contoh :

11001101(2) = … (8) ===> 011 001 101

011 = 3 (lihat tabel konversi di atas)

001 = 1

101 = 5

Hasil Konversi : 315 (ditulis dari atas ke bawah)

5. Biner ke Desimal

Cara mengkonversi bilangan biner ke desimal adalah dengan cara mengalikan satu persatu bilangan dengan 2 (basis bilangan biner) pangkat 0, pangkat 1 dan seterusnya sesuai dengan banyaknya bilangan biner yang akan di konversi dan perhitungannya dimulai dari bilangan biner yang paling kanan.

Contoh :

$00011_{(2)} = \ldots_{(10)}$

$= (1x2^0) + (1x2^1) + (0x2^2) + (0x2^3) + (0x2^4)$

$= 1 + 2 + 0 + 0 + 0$

$= 3$

Hasil Konversi : 3

6. Biner ke Hexadecimal

Cara mengkonversi bilangan biner ke hexadesimal tekniknya hampir sama dengan cara konversi bilangan biner ke oktal. **Yang membedakan ada pada pengelompokan bilangan binernya, pada bilangan oktal dalam satu kelompok terdiri dari 3 bit bilangan biner, sedangkan pada hexadesimal dalam satu kelompok terdiri dari 4 bit bilangan biner**.

Contoh :

$10100_{(2)}$ = ... (16)

· 0001 = 1 (lihat tabel konversi di atas)

· 0100 = 4

Hasil Konversi = 14 (ditulis dari atas ke bawah)

7. Octal Ke Desimal

Cara mengkonversi bilangan oktal ke desimal adalah dengan cara mengalikan satu persatu bilangan dengan 8 (basis bilangan oktal) dengan pangkat 0, 1 dan seterusnya dimulai dari bilangan oktal yang paling kanan. Kemudian hasil dari semua pengalian dijumlahkan.

Contoh :

$62_{(8)}$ = ... (10)

· $2x8^0 = 2$

· $6x8^1 = 48$

2+48 = 50

Hasil Konversi = 50

8. Octal Ke Biner

Cara mengkonversi bilangan oktal ke biner adalah dengan **memecah** terlebih dahulu bilangan oktal kedalam satuan bilangan, kemudian masing-masing bilangan diubah kedalam bentuk 3 bit bilangan biner, dengan cara membagi setiap satuan bilangan tersebut dengan 2 (basis bilangan biner). Jika hasil konversi hanya menghasilkan 2 digit bilangan biner, maka harus ditambahkan 0 di sebelah kirinya, supaya bilangan binernya menjadi 3 digit (bit).
Contoh :

$145_{(8)}$ = ... (2)

1 = 001 (lihat tabel konversi)

4 = 100

5 = 101

Hasil Konversi = 001100101 (ditulis dari atas ke bawah)

9. Octal Ke Hexadecimal

Cara mengkonversi bilangan oktal ke hexadesimal terdiri dari dua tahap yaitu:

- Pertama, mengkonversi terlebih dahulu setiap bit bilangan oktal ke bilangan biner

- Kedua, hasil konversi ke bilangan biner kemudian di konversikan ke bilangan hexadesimal

Singkatnya seperti ini: Oktal --> Biner --> Hexadesimal.

Contoh :

$46_{(8)}$ = ... (16)
Diubah ke Biner

4 = 100

6 = 110

Diubah Ke Hexadecimal

0100 = 2

0110 = 6

Hasil Konversi = 26

10. Hexadecimal ke Biner

Sama dengan cara konversi bilanga octal ke biner, **bedanya kalau bilangan octal binernya harus 3 bit, sedangkan bilangan hexadesimal binernya terdiri dari 4 bit**. Misal kita konversi 2 hexa menjadi biner hasilnya bukan 10 melainkan 0010
Contoh :

$9A_{(16)}$ = ... (2)
9 = 1001 (lihat tabel konversi)

A/10 = 1010

Hasil Konversi = 10011010 (ditulis dari atas)

11. Hexadecimal ke Desimal

Cara mengkonversi bilangan biner ke desimal adalah dengan mengalikan satu-satu bilangan dengan 16 (basis hexa) pangkat 0 atau 1 atau 2 dst., dimulai dari bilangan paling kanan. Kemudian hasilnya dijumlahkan.

Contoh :

$7A_{(16)} = \ldots_{(10)}$

$= (7x16^1) + (10x16^0)$

$= 112 + 10$

$= 122$

Hasil Konversi = 122

12. Hexadecimal ke Oktal

Begitu juga dengan konversi hexa desimal ke octal yakni dengan mengubah bilangan hexa ke bilangan biner, kemudian diubah menjadi bilangan octal. Ringkasnya: hexa->biner->octal

Contoh :

$AF_{(16)} = \ldots_{(10)}$

Diubah ke Biner

A/10 = 1010 (lihat tabel konversi)

F/15 = 1111 ==> hasil konversinya 10101111

Diubah ke Oktal ==> 010 101 111

010 = 2 (lihat tabel konversi)

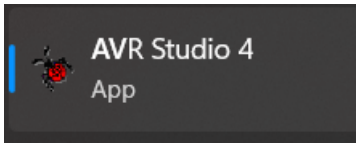
101 = 5

111 = 7

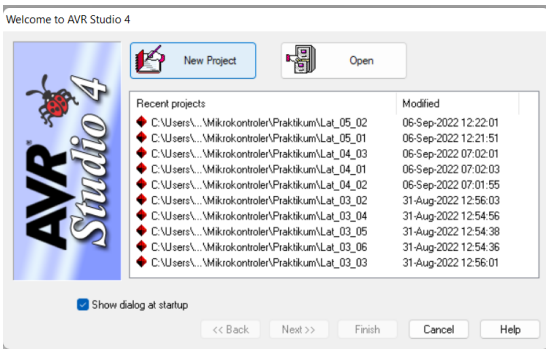
Hasil Konversi = 257 (ditulis dari atas)

C. LANGKAH KERJA

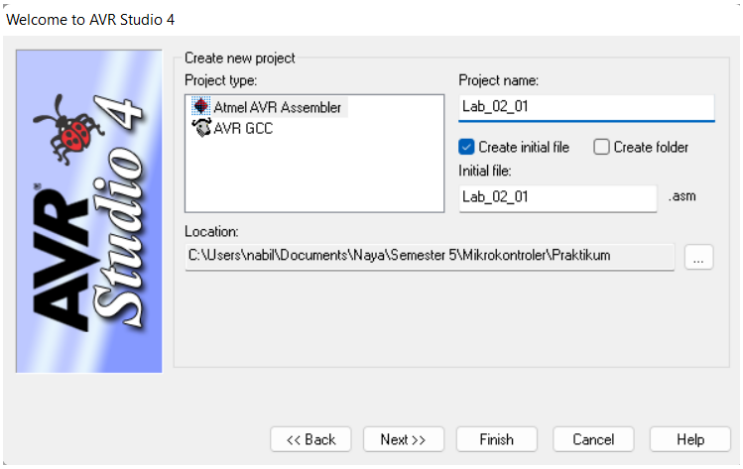
1. *Install* AVR Studio 4
2. Membuka aplikasi

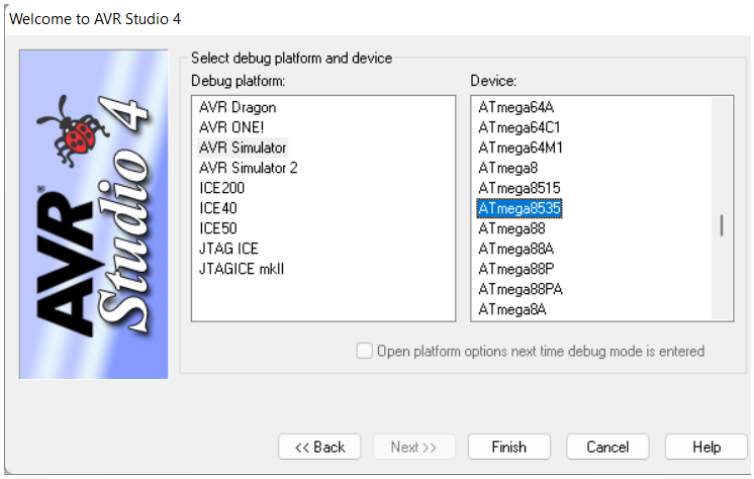


3. Membuat *project* baru

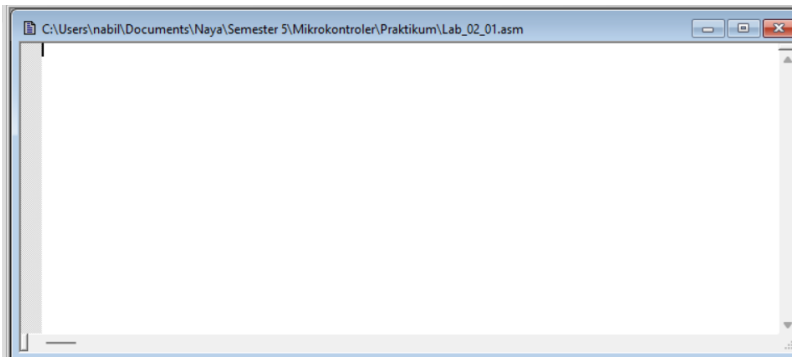


4. Memberikan nama pada project tersebut → klik **next** → klik **AVR simulator** → klik **ATmega8535** → klik **Finish**





5. Menuliskan program pada *sheet* yang disediakan.



6. Kemudian, lakukan *running* program

 , F11

D. PEMBAHASAN DAN ANALISIS

1. Buat program konversi data desimal 2 digit (yang ada di r20) ke oktal simpan hasil konversi pada r22:r21.

    a. Cara 1

- Kode Program

```
.INCLUDE "M8535DEF.INC"
.ORG 0X00
RJMP MAIN


MAIN:LDI R27, LOW (RAMEND); INISIALISASI STACK
    OUT SPL, R27
    LDI R27, HIGH (RAMEND)
    OUT SPH, R27
      LDI r20,0x99
      MOV R21,R20
      ANDI R21,0XF0
      LSR R21
      LSR R21
      LSR R21
      LSR R21
      LDI R23,10
      MUL R21,R23
      MOV R21,R0
      MOV R22,R20
      ANDI R22,0X0F
      ADD R21,R22
```

```
MOV R24,R21
ANDI R24,0X7
MOV R25,R21
ANDI R25,0XF8
ADD R25,R25
ADD R25,R24

MOV R26,R25
ANDI R26,0X7F
MOV R27,R25
ANDI R27,0X80
ADD R27,R27
ADC R28,R1
ADD R27,R26
ADC R28,R1

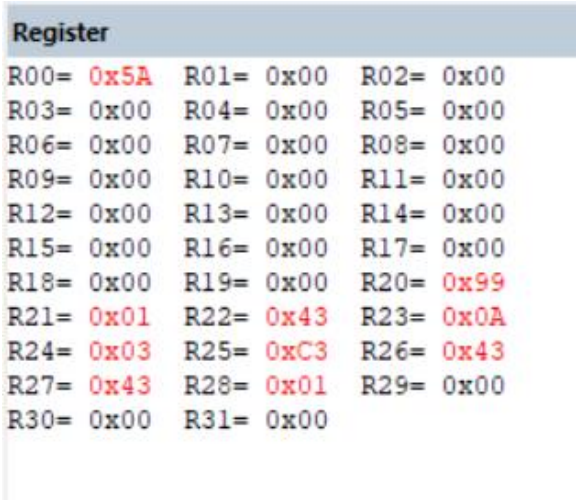
MOV R21,R1
MOV R22,R3
MOV R21,R28
MOV R22,R27



MOV R21,R1
MOV R22,R3
MOV R21,R28
MOV R22,R27
```

```
AKHIR: RJMP AKHIR
```

- Hasil Program



b. Cara 2

- Kode Program

```
.include "m8535def.inc"

.org 0x00

rjmp main

        ; PAKAI PROJECT "ROOT"

main :

ldi r20,low(ramend)

out spl,r20

ldi r20,high(ramend)

out sph,r20


ldi r20,0x69            ; register input data

ldi r18,0x0a            ; faktor pengali 0A
```

```
mov r16,r20
andi r16,0xf0            ; mengambil nilai puluhan
lsr r16
lsr r16
lsr r16
lsr r16
mul r16,r18                    ; puluhan kali 0A
mov r16,r20
andi r16,0x0f
add r0,r16
mov r17,r0
andi r17,0x07  ;mengambil 3 LSB terkahir; awal proses penyisipan 0 pertama
mov r21,r17
mov r17,r0
andi r17,0xf8            ; mengambil 5 MSB awal
mov r25,r17
add r25,r25
add r25,r21                    ; akhir proses penyisipan 0 pertama


mov r17,r25
andi r17,0x80            ; mengambil 1 MSB
mov r21,r17
mov r17,r25
andi r17,0x7f            ; mengambil 7 Bit terkahir
mov r23,r17
```

```
add r21,r21

adc r22,r31

add r21,r23


akhir: rjmp akhir
```

- Hasil Program



c. Cara 3

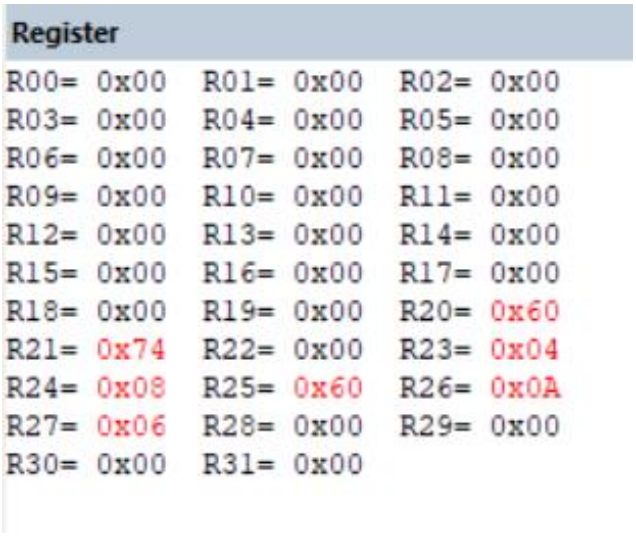
- Kode Program

```
.include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
main :
ldi r20,low(ramend)
out spl,r20
ldi r20,high(ramend)
out sph,r20
;====================
ldi r20,0x60
L0:     inc r21                         ; Kesatuan Program Up Counter Oktal
```

```
        mov r23,r21
        andi r23,0x0f
        cpi r23,8
        brlt L1
        ldi r24,8                    ; Secara Oktal 8 + 8 = 10
        add r21,r24
;====================
L1: INC R25
        MOV R26,R25
        ANDI R26,0X0F
        CPI R26,0X0A
        BRLT L2
        LDI R27,6
        ADD R25,R27
L2:     CP R25,R20
        BRNE L0
;====================

akhir: rjmp akhir
```

- Hasil Program



d. Cara 4

- Kode Program

```
  .include "m8535def.inc"
.org 0x00
rjmp main
     ; PAKAI PROJECT "ROOT"

MAIN:LDI R27, LOW (RAMEND); INISIALISASI STACK
     OUT SPL, R27
     LDI R27, HIGH (RAMEND)
     OUT SPH, R27

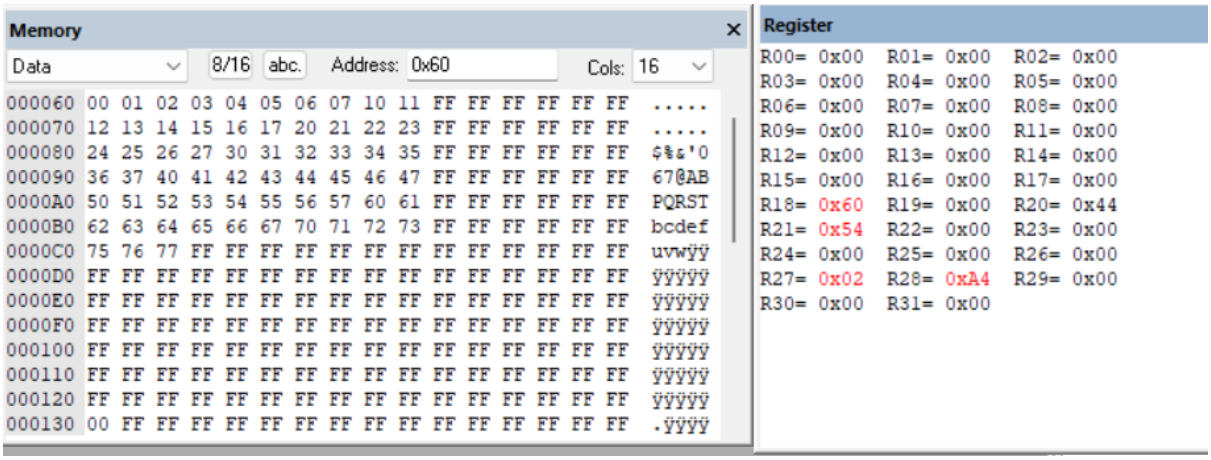
     MOV R16,R20
     ANDI R16,0XF0
     LSR R16
     LSR R16
     LSR R16
     LSR R16
     LDI R23,0X0A
     MUL R16,R23

     MOV R17,R20
     ANDI R17,0X0F
     MOV R16,R0
     ADD R16,R17

     MOV R28,R16
     ADD R28,R28
     ADC R29,R29
     LDI R18,0X60
     ADD R28,R18
     ADC R29,R0
     LD R21,Y+
     LD R22,Y
```

```
akhir:rjmp akhir
```

- Hasil Program



2. Buat program konversi data oktal 2 digit (yang ada di r20) ke desimal simpan hasil konversi pada r21.

- Kode Program

```
.include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
main :
ldi r20,low(ramend)
out spl,r20
ldi r20,high(ramend)
out sph,r20

ldi r20,0x44            ; data input
mov r22,r20
andi r22,0xf0           ; mengambil nilai delapanan
lsr r22
lsr r22
```

```
lsr r22
lsr r22
rcall kal                    ; penggandaan 3 kali
mov r21,r22

mov r22,r20
andi r22,0x0f

add r21,r22
mov r19,r21
rcall daa
mov r21,r19

akhir: rjmp akhir

kal:          ; subrutin perkalian 8 secara desimal = penggandaan secara 3 kali
ldi r16,0x03
l1:     add r22,r22
        mov r19,r22
        rcall daa
        mov r22,r19
        dec r16
        brne l1
ret

daa:    push r16;amankan isi reg. ke stack
                push r17
                push r21
                push r24
                in r16,sreg; amankan sreg
                in r17,sreg
                ldi r24,0 ; r24 hanya sbg status dr carry flag
                adc r24,r24; status carry flag sebelum koreksi
```

```
; ambil, amankan kembali sreg, uji half carry
                out sreg,r17
                brhc lup4; lompat ke lup4 jika H=0
                rcall fk_ln; satuan + 06 jika H=1
        ;       rjmp lup5
lup4:   mov r21,r19
                andi r21,$0f; ambil dgt.satuan
                cpi r21,$0a ; > 9
                brcs lup5; lompat ke lup5 jika < 9
                rcall fk_ln; satuan + 06 jika > 9
lup5:   out sreg,r17; ambil, amankan kembali sreg, uji carry
                brcc lup6; lompat ke lup6 jika C=0
                rcall fk_hn; puluhan + 60 jika C=1
        ;       rjmp lup7
lup6:   mov r21,r19
                andi r21,0xf0; ambil dgt.puluhan
                cpi r21,0xa0
                brcs lup7; lompat ke lup7 jika pul < a
                rcall fk_hn; pul + 60 jika > 9
lup7:   out sreg,r16; mengatur carry flag sblm keluar:
                clc; C = 0
                in r16,sreg ;amankan sreg stlh c=0
                cpi r24,0 ;menguji status
                breq lup8
                out sreg,r16;ambil sreg, set c
                sec; C = 1
                in r16,sreg; amankan sreg stlh c=1
lup8:
                out sreg,r16;status akhir
                pop r24;kembalikan isi reg. dr stack
                pop r21
                pop r17
                pop r16
```

```
                ret
fk_ln:  ldi r21,06; koreksi dgt.satuan
                out sreg,r16
                add r19,r21
                in r16,sreg
                brcc tuh
                ldi r21,0x01
                add r17,r21
tuh:    out sreg,r16
                adc r24,r24
                ret
fk_hn:  ldi r21,$60; koreksi dgt.puluhan
                out sreg,r16
                add r19,r21
                in r16,sreg
                adc r24,r24; amankan carry pd r24
                ret ; akhir dari prog. subrutin.
```

- Hasil Program



3. Buat program konversi data desimal 4 digit (yang ada di r22 : r21) ke oktal simpan hasil konversi pada r25:r24:r23.
   - Kode Program

- Hasil Program

4. Buat program konversi data oktal 4 digit (yang ada di r21:r20) ke desimal simpan hasil konversi pada r23:r22.
   - Kode Program

   - Hasil Program

E. Analisis

Pada praktikum kali ini, terdapat 3 program yang perlu dilakukan, namun sebelumnya ada beberapa perintah yang biasanya selalu ada agar dapar membuat suatu program berjalan. Praktikum kali ini menggunakan *software* simulasi mikrokontroler yaitu ATmega 8535 dan menggunakan konsep bahasa Assembly yang dimana bahasa pemrograman ini termasuk ke bahasa tingkat renda (*low level language*) atau bisa dibilang bahasa mesin. Simulator ATmega 8535 bukan simulator yang *sensitive case* sehingga besar kecilnya huruf ketika penulisan program tidak akan berpengaruh kepada berjalannya suatu program (kecuali suatu nama subrutin, harus sama antara nama subrutin dengan subrutin yang dipanggil).

Dari setiap program yang digunakan, terdapat beberapa baris perintah "wajib" agar program dapat berjalan. Penjelasan dari tiap baris program yang digunakan tersebut sebagai berikut:

Perintah/Sintaks	**Deskripsi**
.include"m8535def.inc"	Mendefinisikan mikrokontroler yang direferensikan untuk program yaitu ATmega8535.
.org 0x0000	Mendefinisikan alamat awal program.

rjmp main	Melakukan *jump* (*relative jump*) ke label main *procedure* (prosedur utama).
ldi r16,low(ramend) out spl,r16 ldi r16,high(ramend) out sph,r16	Menyiapkan *stack* (*stack pointer low* dan *stack pointer high*) untuk menerima data-data yang akan diletakkan/digunakan. Menempatkan 8bit pertama dan beberapa bit kedua ke R16, "low" berarti 8bit pertama di sebelah kanan dan "high" berarti beberapa bit kedua. "ramend" berarti alamat ram terakhir.
Akhir : rjmp Akhir	Untuk mengakhiri proses *running* programnya.

F. KESIMPULAN

Pada praktikum yang telah dilakukan, diketahui bahwa simulator yang digunakan yaitu ATmega 8535, diketahui pula cara kerjanya, serta memahami konsep dari bahasa pemograman Assembly. Pada praktikum ini pula, diketahui bahwa konversi suatu data desimal ke oktal menggunakan 4 cara. Cara pertama dengan menyisipkan nilai 0 pada setiap 4 bit, cara kedua dengan menggunakan K'10, cara ketiga dengan menggunakan 2 counter, yaitu counter 1 (menghitung turun secara desimal) dan counter 2 (menghitung naik secara oktal), dan cara keempat dengan menggunakan metode Lookup Table yang disimpat di data area. Kemudian untuk mengonversi oktal ke desimal harus dibagi dengan 8 hingga habis kemudian dujumlahkan dengan nilai satuannya. Pada simulator ATmega 8535 tidak dapat menggunakan penjumlahan secara langsung, sehingga dibuat sebuah subrutin untuk menjumlahkan secara desimal. Kemudian, digunakan konsep pengambilan nilai suatu puluhannya saja, atau satuan, dan lainnya menggunakan perintah **ANDI** dan **LSR.** Kemudian, terdapat perintah **ADD** untuk menjumlahkan datanya dan **ADC** untuk menjumlahkan yang terdapat *carry*nya.

DAFTAR PUSTAKA

[1] Admin, "Sistem Komputer," 21 Mei 2013. [Online]. Available: http://sistemkomputer.narotama.ac.id/2013/05/mikrokontroler-atmega8535/. [Accessed 14 September 2022].

[2] C. Tjahyadi, "AVR Studio dan WinAVR," [Online]. Available: http://christianto.tjahyadi.com/belajar-mikrokontroler/avr-studio-dan-winavr.html. [Accessed 14 September 2022].